

Day 7 — SIEM Log Normalisation & Parsing

Name: Mohammed Farhan
Role: Associate – Information Security (Intern)
Organization: CyberShelter UAE
Date: 5 July 2026

Objective

To understand how SIEMs parse unstructured log data by writing a custom Wazuh decoder and detection rule for a fictional application log format, write and test a Logstash Grok pattern for nginx logs, ship one log type through the complete ELK pipeline (Filebeat → Logstash → Elasticsearch → Kibana), and consolidate all Week 1 deliverables into a submittable package.

System Information

Detail	Value
Host	Ubuntu Server 22.04 LTS — 192.168.56.104
Wazuh	Manager/Indexer/Dashboard (Docker)
ELK Stack	Elasticsearch/Logstash/Kibana 9.4.2 (Docker)
Filebeat	9.4.2, nginx + auditd modules

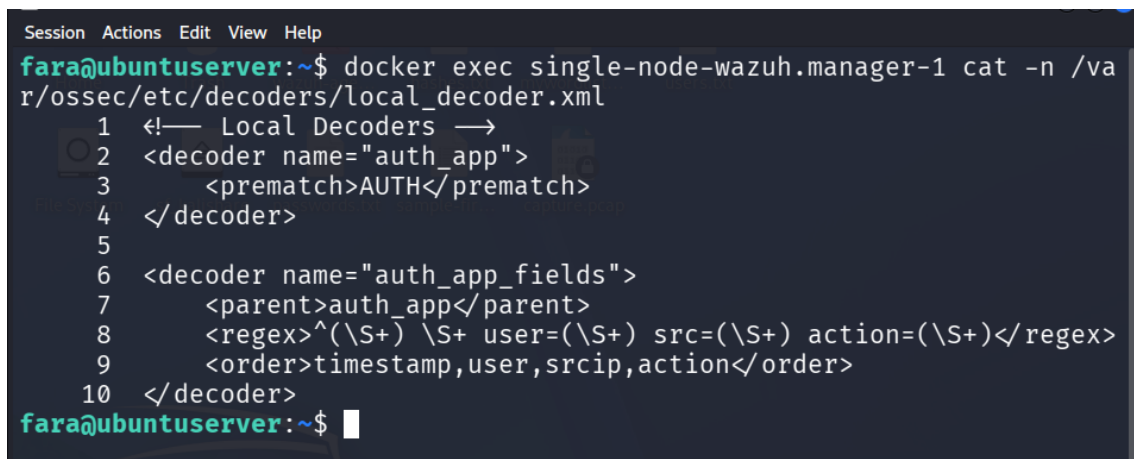
Part 1 — Custom Wazuh Decoder

Target log format (fictional application):

2026-06-15T10:22:01 [AUTH] user=admin src=192.168.1.50 action=LOGIN_FAILED

Step 1 — Created the Decoder File on Ubuntu Server

- `cat > ~/local_decoder.xml << 'EOF'`
- `<!-- Local Decoders -->`
- `<decoder name="auth_app">`
- `<prematch>AUTH</prematch>`
- `</decoder>`
- `<decoder name="auth_app_fields">`
- `<parent>auth_app</parent>`
- `<regex>^(\\S+) \\S+ user=(\\S+) src=(\\S+) action=(\\S+)</regex>`
- `<order>timestamp,user,srcip,action</order>`
- `</decoder>`
- `EOF`



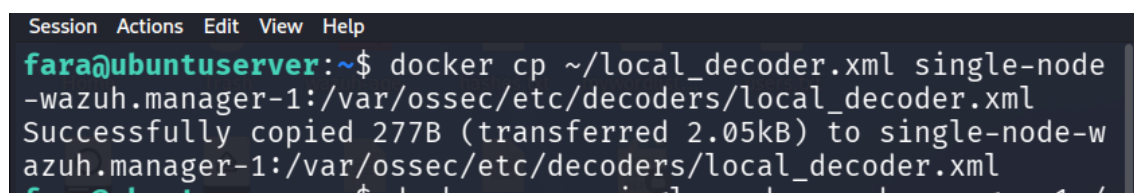
```
Session Actions Edit View Help
fara@ubuntu-server:~$ docker exec single-node-wazuh.manager-1 cat -n /var/ossec/etc/decoders/local_decoder.xml
1  <!-- Local Decoders -->
2  <decoder name="auth_app">
3      <prematch>AUTH</prematch>
4  </decoder>
5
6  <decoder name="auth_app_fields">
7      <parent>auth_app</parent>
8      <regex>^(\\S+) \\S+ user=(\\S+) src=(\\S+) action=(\\S+)</regex>
9      <order>timestamp,user,srcip,action</order>
10 </decoder>
fara@ubuntu-server:~$
```

What this does:

Creates the decoder file directly on Ubuntu Server's home directory using a heredoc (writes multiple lines to a file in one command).

Step 2 — Copy the File Into the Wazuh Manager Container

- `docker cp ~/local_decoder.xml single-node-wazuh.manager-1:/var/ossec/etc/decoders/local_decoder.xml`



```
Session Actions Edit View Help
fara@ubuntu-server:~$ docker cp ~/local_decoder.xml single-node-wazuh.manager-1:/var/ossec/etc/decoders/local_decoder.xml
Successfully copied 277B (transferred 2.05kB) to single-node-wazuh.manager-1:/var/ossec/etc/decoders/local_decoder.xml
fara@ubuntu-server:~$ docker exec single-node-wazuh.manager-1 cat /var/ossec/etc/decoders/local_decoder.xml
```

What this does: Copies the file from Ubuntu Server into Wazuh Manager's actual decoder folder inside the Docker container.

Step 3 — Restart Wazuh Services to Load the New Decoder

- `docker exec single-node-wazuh.manager-1 /var/ossec/bin/wazuh-control start`

```
fara@ubuntu-server:~$ docker exec single-node-wazuh.manager-1 /
var/ossec/bin/wazuh-control start
2026/07/06 07:56:27 wazuh-modulesd:router: INFO: Loaded router
module.
2026/07/06 07:56:27 wazuh-modulesd:content_manager: INFO: Load
ed content_manager module.
Starting Wazuh v4.9.0 ...
wazuh-apid already running ...
Started wazuh-csyslogd ...
Started wazuh-dbd ...
2026/07/06 07:56:28 wazuh-integratord: INFO: Remote integratio
ns not configured. Clean exit.
Started wazuh-integratord ...
Started wazuh-agentlessd ...
wazuh-authd already running ...
wazuh-db already running ...
wazuh-execd already running ...
wazuh-analysisd already running ...
wazuh-syscheckd already running ...
wazuh-remoted already running ...
wazuh-logcollector already running ...
wazuh-monitor already running ...
wazuh-modulesd already running ...
```

What this does: Restarts all Wazuh internal services (analysisd, remoted, etc.) so the new decoder file is read and loaded into the active ruleset.

Step 5 — Test the Decoder Using wazuh-logtest (Field Extraction)

- `docker exec -it single-node-wazuh.manager-1 /var/ossec/bin/wazuh-logtest`

Then paste the test log line:

2026-06-15T10:22:01 [AUTH] user=admin src=192.168.1.50 action=LOGIN_FAILED

```
fara@ubuntu-server:~$ docker exec -it single-node-wazuh.manager
-1 /var/ossec/bin/wazuh-logtest
Starting wazuh-logtest v4.9.0
Type one log per line

2026-06-15T10:22:01 [AUTH] user=admin src=192.168.1.50 action=
LOGIN_FAILED
```

Result-(Decoding) confirmed all four fields extracted:

- timestamp: '2026-06-15T10:22:01'
- dstuser: 'admin'
- srcip: '192.168.1.50'
- action: 'LOGIN_FAILED'

```
**Phase 1: Completed pre-decoding.  
    full event: '2026-06-15T10:22:01 [AUTH] user=admin src  
=192.168.1.50 action=LOGIN_FAILED'  
  
**Phase 2: Completed decoding.  
    name: 'auth_app'  
    action: 'LOGIN_FAILED'  
    dstuser: 'admin'  
    srcip: '192.168.1.50'  
    timestamp: '2026-06-15T10:22:01'
```

Part 2 — Custom Detection Rule (Frequency-Based)

Create rules file:

- cat > ~/local_rules.xml << 'EOF'
- <!-- Local Rules -->
- <group name="auth_app,">
- <rule id="100100" level="5">
- <decoded_as>auth_app</decoded_as>
- <match>LOGIN_FAILED</match>
- <description>Custom app: failed login attempt for user
\$(dstuser)</description>
- </rule>
- <rule id="100101" level="10" frequency="5" timeframe="60">
- <if_matched_sid>100100</if_matched_sid>
- <same_source_ip />
- <description>Custom app: 5+ failed login attempts from
\$(srcip) in 60 seconds - possible brute force</description>
- </rule>
- </group>
- EOF

```
fara@ubuntu:~$ cat > ~/local_rules.xml << 'EOF'
<!-- Local Rules -->
<group name="auth_app,">

  <rule id="100100" level="5">
    <decoded_as>auth_app</decoded_as>
    <match>LOGIN_FAILED</match>
    <description>Custom app: failed login attempt for user $(dstuser)</description>
  </rule>

  <rule id="100101" level="10" frequency="5" timeframe="60">
    <if_matched_sid>100100</if_matched_sid>
    <same_source_ip />
    <description>Custom app: 5+ failed login attempts from $(srcip) in 60 seconds
    le brute force</description>
  </rule>

</group>
EOF
```

Copy into container:

- `docker cp ~/local_rules.xml single-node-wazuh.manager-1:/var/ossec/etc/rules/local_rules.xml`

Restart Wazuh services:

- `docker exec single-node-wazuh.manager-1 /var/ossec/bin/wazuh-control start`

Step 7 — Test the Frequency Rule

```
fara@ubuntu:~$ docker exec -it single-node-wazuh.manager-1 /var/ossec/bin/wazuh-logtest
Starting wazuh-logtest v4.9.0
Type one log per line

2026-06-15T10:22:01 [AUTH] user=admin src=192.168.1.50 action=LOGIN_FAILED
```

Submit the same test log line **5 times in a row** via wazuh-logtest:

```
**Phase 1: Completed pre-decoding.  
full event: '2026-06-15T10:22:01 [AUTH] user=admin src=192.168.1.50 action=LOGIN_FAILED'  
  
**Phase 2: Completed decoding.  
name: 'auth_app'  
action: 'LOGIN_FAILED'  
dstuser: 'admin'  
srcip: '192.168.1.50'  
timestamp: '2026-06-15T10:22:01'  
  
**Phase 3: Completed filtering (rules).  
id: '100100'  
level: '5'  
description: 'Custom app: failed login attempt for user admin'  
groups: '['auth_app']'  
firedtimes: '1'  
mail: 'False'  
**Alert to be generated.
```

```
2026-06-15T10:22:01 [AUTH] user=admin src=192.168.1.50 action=LOGIN_FAILED  
  
**Phase 1: Completed pre-decoding.  
full event: '2026-06-15T10:22:01 [AUTH] user=admin src=192.168.1.50 action=LOGIN_FAILED'  
  
**Phase 2: Completed decoding.  
name: 'auth_app'  
action: 'LOGIN_FAILED'  
dstuser: 'admin'  
srcip: '192.168.1.50'  
timestamp: '2026-06-15T10:22:01'  
  
**Phase 3: Completed filtering (rules).  
id: '100100'  
level: '5'  
description: 'Custom app: failed login attempt for user admin'  
groups: '['auth_app']'  
firedtimes: '2'  
mail: 'False'  
**Alert to be generated.
```

```
2026-06-15T10:22:01 [AUTH] user=admin src=192.168.1.50 action=LOGIN_FAILED  
  
**Phase 1: Completed pre-decoding.  
full event: '2026-06-15T10:22:01 [AUTH] user=admin src=192.168.1.50 action=LOGIN_FAILED'  
  
**Phase 2: Completed decoding.  
name: 'auth_app'  
action: 'LOGIN_FAILED'  
dstuser: 'admin'  
srcip: '192.168.1.50'  
timestamp: '2026-06-15T10:22:01'  
  
**Phase 3: Completed filtering (rules).  
id: '100100'  
level: '5'  
description: 'Custom app: failed login attempt for user admin'  
groups: '['auth_app']'  
firedtimes: '3'  
mail: 'False'  
**Alert to be generated.
```

```

2026-06-15T10:22:01 [AUTH] user=admin src=192.168.1.50 action=LOGIN_FAILED

**Phase 1: Completed pre-decoding.
full event: '2026-06-15T10:22:01 [AUTH] user=admin src=192.168.1.50 action=LOGIN_FAILED'

**Phase 2: Completed decoding.
name: 'auth_app'
action: 'LOGIN_FAILED'
dstuser: 'admin'
srcip: '192.168.1.50'
timestamp: '2026-06-15T10:22:01'

**Phase 3: Completed filtering (rules).
id: '100100'
level: '5'
description: 'Custom app: failed login attempt for user admin'
groups: '['auth_app']'
firedtimes: '4'
mail: 'False'
**Alert to be generated.

```

```

2026-06-15T10:22:01 [AUTH] user=admin src=192.168.1.50 action=LOGIN_FAILED

**Phase 1: Completed pre-decoding.
full event: '2026-06-15T10:22:01 [AUTH] user=admin src=192.168.1.50 action=LOGIN_FAILED'

**Phase 2: Completed decoding.
name: 'auth_app'
action: 'LOGIN_FAILED'
dstuser: 'admin'
srcip: '192.168.1.50'
timestamp: '2026-06-15T10:22:01'

**Phase 3: Completed filtering (rules).
id: '100101'
level: '10'
description: 'Custom app: 5+ failed login attempts from 192.168.1.50 in 60 seconds - possible brute force'
groups: '['auth_app']'
firedtimes: '1'
frequency: '5'
mail: 'False'
**Alert to be generated.

```

Attempt	Rule Fired	Level
1	100100	5
2	100100	5
3	100100	5
4	100100	5
5	100101	10

Result:

Rule 100101 fired on the 5th attempt with description: "5+ failed login attempts from 192.168.1.50 in 60 seconds - possible brute force" — confirming the frequency-based detection works correctly.

Part 3 — Logstash Grok Pattern

Target log format: nginx access log (Common Log Format)

```
127.0.0.1 - - [06/Jul/2026:15:41:18 +0400] "GET /test-day7 HTTP/1.1"
404 162 "-" "curl/7.81.0"
```

Final working Grok filter (~/.docker-elk/logstash/pipeline/logstash.conf):

```
fara@ubuntuuserver:~/docker-elk$ cat ~/.docker-elk/logstash/pipeline/logstash.conf
input {
  beats {
    port => 5044
  }
  tcp {
    port => 50000
  }
}

filter {
  if [event][dataset] == "nginx.access" or [log][file][path] =~ "nginx" {
    grok {
      match => {
        "message" => '%{IP:client_ip} - - \[%{HTTPDATE:timestamp}\] "%{WORD:method} %{URIPATHPARAM:url} HTTP/%{NUMBER:http_version}" %{NUMBER:status} %{NUMBER:bytes}'
      }
    }
  }
}

output {
  elasticsearch {
    hosts => "elasticsearch:9200"
    user => "logstash_internal"
    password => "${LOGSTASH_INTERNAL_PASSWORD}"
    index => "logs-generic-default"
  }
}
```

Tested in grokdebugger

Grok Pattern	Output
<pre>%{IP:client_ip} - - \[%{HTTPDATE:timestamp}\] "%{WORD:method} %{URIPATHPARAM:url} HTTP/%{NUMBER:http_version}" %{NUMBER:status} %{NUMBER:bytes}</pre>	<pre>[{ "client_ip": "127.0.0.1", "timestamp": "05/Jul/2026:15:13:", "method": "HEAD", "url": "/api/users", "http_version": "1.1", "status": "404", "bytes": "0" }]</pre>
Samples <pre>1 127.0.0.1 - - [05/Jul/2026:15:13:24 +0400] "HEAD /api/users HTTP/1.1" 404 0 "-" "curl/7.81.0"</pre>	

Part 4 — Full Pipeline: Filebeat → Logstash → Elasticsearch → Kibana

Pipeline reconfiguration:

Filebeat's output was switched from direct Elasticsearch shipping (used on Day 6) to output.logstash, pointing at port 5044 — inserting Logstash as a parsing layer in the pipeline

A conditional check (if [event][dataset] == "nginx.access") was added to the Grok filter so it only processes nginx events, leaving other log types (auditd) untouched

Verification — Parsed Document in Kibana:

Raw log (preserved in event.original):

```
127.0.0.1 - - [06/Jul/2026:15:41:18 +0400] "GET /test-day7 HTTP/1.1"
404 162 "-" "curl/7.81.0"
```



Pattern:

Documents (1)	Patterns (1)	Field statistics
	Count ↑ Log rate	Examples
	> 1	127.0.0.1 - - [06/Jul/2026:15:41:18 +0400] "GET /test-day7 HTTP/1.1" 404 162 "-" "curl/7.81.0"

Parsed structured fields (confirmed in Kibana Discover):

client_ip: 127.0.0.1

method: GET

url: /test-day7

status: 404

<pre>], "log.file.path.text": ["/var/log/nginx/access.log"], "agent.name": ["ubuntuserver"], "client_ip": ["127.0.0.1"] } </pre>	<pre> "message": ["127.0.0.1 - - [06/Jul/2026:15:41:18 +0400] \"GET /test-day7 HTTP/1.1\" 404 162 \"-\" \"curl/7.81.0\""], "data_stream.type": ["logs"], "url": ["/test-day7"] </pre>
---	---

<pre> "host.os.type": ["linux"], "method": ["GET"], "data_stream.namespace": ["default"] </pre>	<pre> "agent.type": ["filebeat"], "event.module": ["nginx"], "agent.name.text": ["ubuntuserver"] </pre>
---	---

This confirms the complete pipeline: nginx generates a raw log line → Filebeat ships it → Logstash's Grok filter parses it into six structured fields → Elasticsearch stores it → Kibana displays both the raw and parsed versions side by side.

Conclusion

Day 7 focused on building a custom log parser from scratch, testing the real mechanics behind Wazuh detection rules rather than using pre-built ones